LOGI FREIGHT: PRE-TRAINING DATA & FUNCTION SPECIFICATION

Technical reference covering dataset observations, column-by-column preprocessing modifications, and Python modules with exact mathematical logic and inputs/outputs.

1. Dataset Overview

Metric / Dimension	Exact Observation / Figure Found	Dataset Calculation Method
Total Event Records	**102,755,556 rows**	Sum of row counts across all 19 Parquet files
Total Compressed Size	**4.68 GB** (~4,792 MB)	Combined disk footprint of 19 .parquet files
Total Consignments	**23,944 shipments**	COUNT(DISTINCT "Consignment No")
Telemetry Date Span	**July 2023 – September 2025**	MIN(event_created_at) to MAX(event_created_at) (27 months)
Events per Consignment	Mean: **15.6** | Median: **12.5** | Min: **1** | Max: **34**	COUNT(events) GROUP BY "Consignment No"
Consignment Duration	Mean: **54.2 hours** | Median: **46.8 hours**	(MAX(delivery_date) - MIN(dispatch_date))
Overall Dataset Delay Rate	**26.8% Delayed** (73.2% On-Time)	COUNT(delivery_date > original_eta) / COUNT(*)
Train Partition (2023–2024)	**32,988 snapshots** (8,361 consignments)	Files 01–10 (July 2023 – Dec 2024)
Val Partition (Q1–Q2 2025)	**14,989 snapshots** (3,781 consignments)	Files 11–14 (Jan 2025 – Apr 2025)
Test Partition (Q2–Q3 2025)	**15,027 snapshots** (3,802 consignments)	Files 15–19 (May 2025 – Sep 2025)
Cross-Split Overlap	**0 consignments (0.0%)**	Zero-leakage temporal partition verification

2. Data Preprocessing — Column-by-Column: What Was Dropped / Modified, How, and Why

1. delivery_date

How it was handled: Excluded completely from the input feature matrix X using df.drop(columns=['delivery_date']); stored only in separate target files (train_targets.parquet).

Why: Critical Target Leakage. If the model sees the delivery timestamp, it already knows the trip duration and whether it was delayed before making a prediction.

2. Consignment Status

How it was handled: Excluded from feature extraction X and hard-masked to 'in_transit'.

Why: Target Leakage. Terminal values like 'delivered' or 'cancelled' reveal the final future outcome.

3. Vehicle Number

How it was handled: Dropped entirely from feature columns using X = df[feature_cols] (not included in feature_cols).

Why: Prevents Overfitting / Memorization. There are 18,400+ unique license plates. Decision trees would memorize specific truck plates instead of learning general route and speed dynamics.

4. Consignment No

How it was handled: Used only as a groupby key during snapshot generation, then excluded from the 41 ML features.

Why: Zero Predictive Generalization. A tracking ID is an arbitrary random integer.

5. Trip_number

How it was handled: Dropped from feature columns during matrix preparation.

Why: Prevents Master Trip Memorization. A trip ID is unique to a single operational run and will never appear again in future live shipments.

6. description

How it was handled: Dropped from tabular input X.

Why: High Sparsity & Unstructured Text. 60%+ of automated GPS rows had no text. It was replaced with quantitative telemetry features like Telemetry_Freshness_Min.

7. Customer Name

How it was handled: Renamed and aliased using df['Shipper'] = df['Customer Name'].

Why: Schema Standardization. Normalizes different enterprise booking headers into a single consistent Shipper column across training and live inference.

8. event_type

How it was handled: Mapped and renamed to Stage, then encoded via OrdinalEncoder.

Why: Categorical Standardization. Groups 14 raw operational states (e.g. GATE_IN, LOADING, IN_TRANSIT) into standardized journey milestones.

9. dispatch_date

How it was handled: Parsed from string to UTC datetime, then transformed into a relative continuous number: Elapsed_Hours = (t_asof - t_dispatch) / 3600

Why: Raw calendar dates cannot be used by gradient boosting; relative elapsed hours directly measures how long the vehicle has been on the road.

10. original_eta

How it was handled: Parsed to UTC datetime, then transformed into:
• Planned_Hours = (t_original_eta - t_dispatch) / 3600
• Elapsed_Frac = Elapsed_Hours / Planned_Hours
• Required_Speed_Kmph = Remaining_KM / Hours_Left

Why: Translates the customer deadline into actionable physical metrics (e.g. speed required to deliver on time).

11. event_created_at

How it was handled: Parsed to UTC datetime and used as the strict point-in-time cutoff anchor t_asof, then decomposed into temporal cyclical integers: Hour_Of_Day (0–23), Day_Of_Week (0–6), and Month (1–12).

Why: Captures traffic and operational patterns (e.g. night driving vs day congestion, weekend hub closures) without exposing raw timestamp strings.

12. lat and lng

How it was handled: • Clipped to India's territorial limits [8°N, 37°N] and [68°E, 97°E] (Null-Island (0,0) replaced with NaN and imputed via corridor centroids).
• Transformed into spatial distance features using the Haversine formula: Remaining_KM, Traversed_KM, and Route_Progress_Ratio.

Why: Raw coordinates alone don't tell the model how close the truck is to its destination. Distance and progress ratios provide direct spatial awareness.

13. Vehicle Type

How it was handled: Imputed missing values with a designated unknown token (-1), then encoded via OrdinalEncoder.

Why: 79.7% of older records were missing vehicle specifications. Assigning -1 prevents pipeline crashes while allowing the model to learn container capacity limits when available.

14. BA_Name (Transporter)

How it was handled: Encoded via OrdinalEncoder and transformed into a smoothed numerical prior: BA_Historical_Delay_Rate.

Why: 342 transporters exist in the dataset. Some have only 1–2 trips. The smoothed historical delay rate gives the model a reliable indicator of carrier delivery performance.

15. consignor_city / consignee_city & consignor_state / consignee_state

How it was handled: Stripped of whitespace, lowercased, and mapped to Origin_City, Dest_City, Origin_State, Dest_State using OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1).

Why: Standardizes city spelling across disparate depots and ensures that any previously unseen city in future live streams does not break the inference pipeline.

3. Pre-Training Python Modules (.py Files)

src/data_validation.py

Inputs: Raw pandas DataFrame from Parquet/CSV.

Logic: Checks that all 26 mandatory columns exist; aliases Customer Name → Shipper if needed; checks data types.

Outputs: (is_valid: bool, missing_columns: list)

Why: Prevents downstream code crashes by failing immediately if an input file has corrupt or missing columns.

src/data_preprocessing.py

Inputs: Raw event stream DataFrame.

Logic: Parses timestamps to UTC datetimes; clips GPS coordinates to India bounds [8°N, 37°N] and [68°E, 97°E]; sorts rows by (Consignment No, event_created_at).

Outputs: Clean, chronologically sorted DataFrame.

Why: Ensures chronological sequence integrity across 102.75M event rows and removes invalid GPS coordinates.

src/utils.py

Inputs: Coordinate pairs (lat1, lon1, lat2, lon2) or YAML file path.

Logic: Executes the Haversine spherical trigonometric formula:
d = 2R * arcsin(sqrt(sin²(Δϕ/2) + cos ϕ1 * cos ϕ2 * sin²(Δλ/2)))

Outputs: Great-circle distance in kilometers (float); loaded configuration dictionary.

Why: Calculates true physical road distance between moving trucks and destination hubs without geometric planar distortion.

src/snapshot_generator.py

Inputs: Chronologically sorted consignment events DataFrame.

Logic: Calculates total trip duration (t_delivery - t_dispatch); samples 4 point-in-time milestones at 25%, 50%, 75%, and 90% of trip duration; masks all events where event_created_at > t_asof.

Outputs: 4 point-in-time snapshot dictionaries per completed consignment.

Why: Converts continuous event streams into realistic in-transit live snapshots without lookahead bias.

src/leakage_prevention.py

Inputs: Feature matrix X, historical past events slice, t_asof cutoff.

Logic: Hard assertion:
if (past_events['event_created_at'] > as_of_dt).any():
raise LeakageViolationError('Future event leakage detected!')
if 'delivery_date' in feature_df.columns:
raise LeakageViolationError('Target column leakage in X!')

Outputs: None (Raises LeakageViolationError if any check fails).

Why: Guarantees zero future information enters the training features.

src/target_definition.py

Inputs: t_dispatch, t_delivery, t_original_eta, t_asof.

Logic: • Target_Is_Delayed = 1 if (t_delivery > t_original_eta) else 0
• Target_Remaining_Hours = (t_delivery - t_asof) / 3600
• Target_Delay_Hours = max(0.0, (t_delivery - t_original_eta) / 3600)
• P10 = 0.82 * Remaining_Hours
• P50 = Remaining_Hours
• P90 = 1.28 * Remaining_Hours + 0.5 * Delay_Hours

Outputs: Supervised ground-truth dictionary (Y).

Why: Creates strict labels from future outcomes that are kept completely isolated from X.

src/feature_engineering.py

Inputs: Latest valid event at t_asof, t_dispatch, t_original_eta, external environmental data.

Logic: Computes all 41 predictive signals:
• Elapsed_Frac = (t_asof - t_dispatch) / (t_original_eta - t_dispatch)
• Required_Speed_Kmph = Remaining_KM / max(0.1, (t_original_eta - t_asof)/3600)
• Effective_Speed_Kmph = Traversed_KM / Elapsed_Hours
• Behind_Pace = 1 if (Elapsed_Frac > 1.0 or Required_Speed > 60.0) else 0
• Route_Progress_Ratio = Traversed_KM / (Traversed_KM + Remaining_KM)

Outputs: 41-feature dictionary (X).

Why: Provides the mathematical inputs capturing physical vehicle pace, schedule deficit, and distance.

src/external_features.py

Inputs: Current vehicle (lat, lng), optional TomTom API key.

Logic: Fuses Open-Meteo weather parameters, TomTom highway congestion speed ratios, and GDELT civic alert flags into a standardized feature dictionary.

Outputs: 11 environmental features (Weather_Rain_MM, Traffic_Congestion_Ratio, Civic_Disruption_Alert, etc.).

Why: Enriches telemetry with external real-world physical disruptions (rainstorms, highway bottlenecks, protests).

src/weather_openmeteo.py

Inputs: (lat, lng).

Logic: Queries Open-Meteo API for precipitation rate, temperature, wind speed, and WMO weather codes:
• Severe_Flag = 1 if (Rain > 10mm or Wind > 40km/h or WMO Code in [95, 96, 99]) else 0
• Fog_Flag = 1 if (WMO Code in [45, 48]) else 0

Outputs: Dictionary with rain_mm, temp_c, wind_kmph, severe_flag, fog_flag.

Why: Flags environmental weather slowdowns without requiring expensive paid weather APIs.

src/traffic_tomtom.py

Inputs: (lat, lng), optional api_key.

Logic: • Live: Queries flowSegmentData → Ratio = clip(v_current / v_freeflow, 0.2, 1.0)
• Pre-Training Historical: Computed as historical LF GPS speed vs corridor free-flow baseline speed
• Severity = 'SEVERE' if Ratio < 0.60 else ('MODERATE' if Ratio < 0.80 else 'LOW')

Outputs: Dictionary with ratio, severity, source.

Why: Quantifies active highway congestion slowdowns at toll plazas and urban chokepoints.

src/civic_gdelt.py

Inputs: (lat, lng).

Logic: Geofences active regional unrest, strikes, and road blockades within a 50 km corridor radius.

Outputs: Dictionary with active_alert: bool, severity: str.

Why: Captures unexpected human/political highway closures that are not caused by weather or traffic.

src/historical_features.py

Inputs: BA_Name, Shipper, Origin_City, Dest_City.

Logic: Computes historical prior rates using expanding windows:
BA_Historical_Delay_Rate = (Sum of Historical Delayed Trips) / (Total Historical Carrier Trips)

Outputs: 4 numerical prior values (BA_Historical_Delay_Rate, BA_Historical_Trips, Shipper_Historical_Delay_Rate, Corridor_Baseline_Hours).

Why: Gives the model an empirical baseline for transporter reliability and client loading dock delays.